Sem. I 2025-2026

Proiect PATR

Control automat al barierei feroviare

Echipa: Lime&Lemon

Data: 6 ianuarie 2026

Componența echipei și contribuția individuală (în %)

Membrii	A	C	D	PR	CB	e-mail
Țuluca Maria-Laura	50%	40%	60%	50%	200%	maria_laura.tuluca@stud.acs.upb.ro
Palade Mădălina Ioana	50%	60%	40%	50%	200%	madalina.palade@stud.acs.upb.ro
	100%	100%	100%	100%		

A-analiză problemă și concepere soluție, C- implementare cod, D-editare documentație, PR-"proofreading", CB - contribuție individuală totală ([%])

Membrii echipei declară că lucrarea respectă toate regulile privind onestitatea academică. În caz de nerespectare a acestora tema va fi notată cu ***0(zero) puncte***

Cuprins

1 Introducere. Definire problemă **1**

2 Analiza problemei **2**

3 Structura și Soluția de Implementare Propusă **4**

3.1 Definirea Structurii Aplicației 4

3.2 Definirea Soluției în Vederea Implementării 5

3.3 Implementarea soluției 7

4 Testarea aplicației și validarea soluției propuse **16**

4.1 Rezultatele execuției 17

5 Bibliografie **19**

1 Introducere. Definire problemă

Problema aleasă de noi pentru implementare constă în modelarea și controlul automat al unei bariere la o trecere la nivel cu cale ferată, unde pot exista mai multe șine paralele și o barieră comună pentru acestea.

Pe fiecare dintre cele k șine sunt montați câte doi senzori la o distanță cunoscută:

- unul plasat înainte de barieră, care detectează apropierea trenului;
- unul imediat după barieră, care semnalează plecarea trenului din regiunea de control.

Sistemul automat de monitorizare primește semnalele de la acești senzori și, pe baza lor, decide momentul închiderii și deschiderii barierei, respectând un set de condiții de siguranță și temporizare.

Funcționarea sistemului trebuie să asigure următoarele cerințe obligatorii:

- **Siguranță:** bariera trebuie să fie complet închisă atunci când există cel puțin un tren în regiunea de interes;
- **Utilitate:** bariera trebuie să fie deschisă în tot timpul în care traficul rutier este sigur;
- **Timp de reacție controlat:** pentru închidere și deschidere există timpi limitați, notați cu τ_{close} și τ_{open};
- **Sincronizare corectă:** dacă un tren se apropie de la stânga, va părăsi zona prin dreapta și invers;
- **Staționare permisă:** trenul poate rămâne un timp $t \leq \tau_{max}$ în zona de control.

Scopul proiectului este dezvoltarea unui sistem în timp real, bazat pe platforma **Arduino** și **FreeRTOS**, care să gestioneze evenimentele generate de senzori și să controleze mecanic poziția barierei (cu ajutorul unui servo motor), respectând constrângerile temporale impuse.

Motivație: Am ales această temă deoarece sistemele de control pentru barierele la trecerile la nivel cu calea ferată reprezintă o aplicație reală, cu care ne întâlnim frecvent în viața de zi cu zi. Pentru noi a părut interesant să înțelegem cum funcționează aceste mecanisme de siguranță și cum pot fi implementate printr-un sistem în timp real.

2 Analiza problemei

Implementarea are ca scop simularea funcționării unei treceri la nivel cu cale ferată, echipată cu o barieră comună și mai multe linii de tren. Fiecare linie este supravegheată de doi senzori:

- **Senzorul S1**: amplasat înainte de barieră, folosit pentru detectarea apropierii trenului;
- **Senzorul S2**: amplasat după barieră, utilizat pentru detectarea părăsirii zonei de control.

Bariera trebuie să fie complet închisă înainte ca trenul să ajungă efectiv în regiunea de interes(de dinaintea barierei). La detectarea unui semnal de la senzorul S_1, sistemul se asigură că nu există alt tren în regiunea de control printr-un semnal cerut de la senzorul S_2. În acest timp, un tren poate parcurge distanța dintre senzorul S_1 și barieră în timp $t \geq \tau_{\min}$.
Dacă în zona de control nu se găsește un alt tren și a trecut perioada de timp $\tau_{\min}$(presupunem că trenul a ajuns la barieră), sistemul declanșează deschiderea barierei, care se realizează într-un timp τ_{open}. După ce trenul este detectat de senzorul S_2 (semnalizând că a trecut complet), sistemul inițiază închiderea barierei, cu o durată τ_{close}. De asemenea, dacă un tren staționează în regiunea de control pentru un timp $t \leq \tau_{\max}$, îi este comandată plecarea.

Analiza unor secvențe corecte de execuție:

- Secvența 1:
  - sosește un tren pe linia 1.
  - dacă bariera nu este deja închisă, sistemul inițiază închiderea.
  - trenul este detectat de senzorul S_{11} și parcurge distanța dintre S_{11} și barieră în timp $t \geq \tau_{\min}$.
  - trenul ajunge în regiunea de interes, în fața barierei.
  - sistemul inițiază deschiderea barierei, cu durata τ_{open}.
  - trenul este detectat de senzorul S_{21} după ce trece complet.
  - sistemul inițiază închiderea barierei, cu durata τ_{close}.
- Secvența 2:

  - sosesc simultan două trenuri, câte unul pe fiecare linie.
  - trenurile sunt detectate de S_{11} și S_{12}; dacă bariera nu este deja închisă, sistemul inițiază închiderea.
  - trenurile parcurg distanța dintre S_{1i} și barieră în timp $t \geq \tau_{\min}$.
  - sistemul inițiază deschiderea barierei, cu durata τ_{close}.
  - după ce trenurile trec complet, sunt detectate de S_{21} și S_{22}.
  - ulterior, bariera începe să se închidă, cu durata τ_{close}.

- Secvența 3:
  - sosesc trei trenuri: două pe linia 1, unul pe linia 2.
  - două trenuri intră până în zona de control (fiecare pe câte o linie), al treilea așteaptă în coadă pe linia 1.
  - trenurile din zona de control sunt detectate de S_{2i} și bariera stă închisă.
  - după ce trenurile pleacă și sunt detectate de S_{2i} ca nemaiexistând în zona de control, trenul din coadă trece în zona de control aferentă liniei 1, deschizându-se bariera.

- Secvența 4:
  - sosesc două trenuri, câte unul pe fiecare linie, însă în zona de control tocmai ce au intrat alte două trenuri.
  - bariera se închide la detectarea trenurilor de către S_{11} și S_{12}.
  - trenurile din zona de control încă nu trebuie să plece și cele din zona de interes așteaptă plecarea celorlaltor două.
  - bariera se deschide doar după ce toate trenurile din zona de control au plecat, pentru a permite trecerea celor din zona de interes.

Secvențe eronate de execuție, care duc la încălcarea regulii de siguranță:

Secvența 1 (eroare de siguranță): Bariera începe deschiderea înainte ca trenul din zona de control să fie detectat de S_2 că a plecat, ceea ce produce risc de accident.
Secvența 2 (eroare logică/de siguranță): Două trenuri se află simultan în zona de control, dar sistemul deschide bariera după plecarea unuia singur, ceea ce duce la încălcarea regulii de siguranță (al doilea tren încă e în zonă).
Secvența 3 (eroare de timp): Trenul trece îm zona de control, dar bariera nu se închide în timpul τ_{close}.

Secvența 4 (eroare de staționare): Trenul staționează mai mult decât τ_{max} și sistemul începe deschiderea barierei.

3 Structura și Soluția de Implementare Propusă

3.1 Definirea Structurii Aplicației

Aplicația este împărțită în trei task-uri (fire de execuție) distincte, fiecare având o responsabilitate clară. Aceste task-uri comunică exclusiv prin mecanisme de sincronizare oferite de FreeRTOS.

Task-ul Principal (Scheduler)

Acest task este implicit în mediul Arduino/FreeRTOS (în funcția `setup()`).

- **Rol:** Inițializează perifericele aplicației noastre (servo, pini I/O), creează obiectele FreeRTOS (cozi de evenimente și comenzile servo, timere software) și lansează în execuție task-urile `taskSensors`, `taskServo` și `taskControl` cu prioritățile definite.

Task-ul Monitorizare Senzori (`taskSensors`)

- **Prioritate:** cea mai scăzută (1).
- **Rol:** Citește periodic starea celor doi senzori ultrasonici (S1 și S2) la intervalul de 60ms. Detectează schimbările de stare (*fronturi*) ale prezenței trenurilor pe șină.
- **Acțiune:** La detectarea unui eveniment relevant (`EVT_S1_PRESENT`, `EVT_S2_PRESENT`, `EVT_S2_CLEAR`), transmite evenimentul către `taskControl` prin intermediul cozii de evenimente. Ne folosim de aplicarea unei tehnici de **histerezis** pe S2 pentru a preveni fluctuațiile false (*jitter*) în detectarea eliberării zonei.

Task-ul Control Barieră (`taskControl`)

- **Prioritate:** cea mai ridicată (3).
- **Rol:** Reprezintă Mașina de Stări (FSM) a sistemului. Ia decizii de Siguranță și Utilitate pe baza evenimentelor primite și a expirării timerelor software.
- **Acțiune:** Reacționează la evenimentele senzorilor, gestionând logica temporală:

1. **EVT_S1_PRESENT:** Inițiază imediat închiderea barierei (Siguranță) și activează timerul τ_{min} (1s timp de gardă).
2. **EVT_TAU_MIN_EXPIRED:** Verifică starea S2, iar dacă S2 este liber, deschide bariera (Utilitate). Dacă S2 este ocupat, verifică cât timp trebuie să se scurgă până la expirarea timerului τ_{max}.
3. **EVT_S2_PRESENT / EVT_S2_CLEAR:** Gestionează starea prezenței trenului în zona de control, extinzând timpul de închidere dacă este necesar(tren după barieră).
4. **EVT_TAU_MAX_EXPIRED:** Plecarea forțată a trenului din zona de control și deschiderea barierei, cu ignorarea ulterioară a S2 până la eliberarea zonei.

Task-ul Acționare Servo (`taskServo`)

- **Prioritate:** medie (2).
- **Rol:** Primește comenzi de la `taskControl` și realizează mișcarea fizică a servomotorului(deschidere/închidere).
- **Acțiune:** Execută mișcarea barierei între `ANGLE_OPEN` și `ANGLE_CLOSE` într-un mod **liniar** pe durata specificată (τ_{open}/τ_{close}), utilizând `vTaskDelay(pdMS_TO_TICKS(1))` pentru o acționare fină, bazată pe pași de 1ms.

3.2 Definirea Soluției în Vederea Implementării

Soluția a fost implementată pentru un microcontroler din familia AVR (compatibil Arduino Mega 2560), utilizând **FreeRTOS.h** ca framework de programare în timp real. Această abordare ne permite o gestiune precisă a timpilor și a concurenței, esențială pentru satisfacerea cerințelor critice de Siguranță (τ_{close}, τ_{min}) și Utilitate (τ_{open}, τ_{max}).

Planificarea se bazează pe principiul **preempțiunii** (preemptive scheduling), iar prioritățile task-urilor au fost alese pentru a asigura că operațiile critice (controlul barierei) sunt executate imediat ce un eveniment apare.

3.2.1 Mecanisme de Sincronizare și Comunicare

Pentru a evita accesul concomitent la resurse și pentru a asigura o comunicare asincronă, am utilizat următoarele mecanisme FreeRTOS:

- **Cozi de Mesaje (`QueueHandle_t`):**
  - **`qEvents`:** Funcționează ca un canal de comunicare între *Task Sursă* → *Task Control*. Permite senzorilor și callback-urilor timer să notifice `taskControl` fără a aștepta, menținând task-ul de control blocat în `xQueueReceive(...)` până când apare un eveniment, asigurând un consum minim de resurse când sistemul este inactiv.
  - **`qServo`:** Funcționează ca un canal de comunicare între *Task Control* → *Task Servo*. Transmite comenzi structurate (`ServoCmd_t`) către servomotor.
- **Timere Software (`TimerHandle_t`):** Sunt esențiale pentru implementarea cerințelor de timp specificate. Sunt timere one-shot care, la expirare, transmit un eveniment specific în `qEvents` (prin funcțiile de callback).
  - **Timere de Acțiune ($\tau_{open}, \tau_{close}$):** Măsoară timpul necesar mișcării fizice a barierei, iar expirarea lor confirmă Siguranța/Utilitatea.
  - **Timer de Siguranță (τ_{min}):** Implementează cerința de a menține bariera închisă minim 1s (timp de gardă) după detectarea trenului la S1, asigurând stabilitatea și siguranța înainte de a lua decizii ulterioare de deschidere(bazate pe S2).
  - **Timer de Utilitate (τ_{max}):** Permite recuperarea sistemului din starea de staționare a trenului (după `5s`), forțând plecarea trenului și deschiderea barierei pentru a reduce blocajul rutier (Utilitate).
- **VTaskDelay (`vTaskDelay`):** Îl utilizăm exclusiv în `taskSensors` pentru a introduce o întârziere periodică de citire (60ms) și în `taskServo` pentru a asigura mișcarea liniară fină (pași de 1ms), permițând scheduler-ului să preia controlul.

3.2.2 Senzori și Servomotor

- **Senzori (S1 și S2):** Utilizăm senzori ultrasonici (HC-SR04), fiecare necesitând o pereche de pini TRIG/ECHO. Funcția `readDistanceCm` implementează logica de citire a timpului de zbor al undei sonore.
- **Histerezisul:** În `taskSensors`, aplicăm histerezis (HYST_CM = 1.0f) pe semnalul S2. Aceasta creează o zonă moartă între pragul de prezență (5.0cm) și cel de eliberare (6.0cm), prevenind *jitter*-ul și schimbările rapide de stare care ar destabiliza `taskControl`.
- **Actuator (Servomotor):** `taskServo` asigură o acționare **liniară** a barierei. În loc să schimbe unghiul instantaneu, mișcarea este împărțită în pași mici pe durata τ_{open}/τ_{close}, simulând o funcționare lină(funcție liniară).

3.2.3 Vizualizare și Stare

Pentru o vizualizare în timp real a stării sistemului, utilizăm monitorul serial (Serial Monitor) al plăcii Arduino, care funcționează ca log central, afișând stările critice:

- `S1 < 5cm -> INCHID bariera...`: Semnalizează acționarea de Siguranță declanșată de S1.
- `TAU_MIN expirat -> verific S2.`: Confirmă încheierea perioadei de gardă de 1s.
- `-- Bariera CLOSED complet...`: Confirmă atingerea poziției de Siguranță (Conformare cu τ_{close}).
- `Masina a stat TAU_MAX=5s...`: Semnalizează intervenția de Utilitate (Alertă de staționare și deschidere forțată).

3.3 Implementarea soluției

În această secțiune, vom prezenta codul final al aplicației noastre pentru controlul barierei de cale ferată. Comentariile din cod detaliază modul de lucru al fiecărui task și al mecanismelor de sincronizare FreeRTOS, punând accent pe logica de Siguranță (τ_{min}) și Utilitate (τ_{max}).

Codul Sursă al Aplicației (Control Barieră)

```
#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <queue.h>
#include <timers.h>
#include <Servo.h>

// Pini
#define PIN_TRIG_S1 8
#define PIN_ECHO_S1 7
#define PIN_TRIG_S2 10
#define PIN_ECHO_S2 6
#define PIN_SERVO   9

// Parametri
const float PRAG_CM = 5.0f;
const float HYST_CM = 1.0f;           // histerezis (clear la >6cm)
```

```
17
18 // Cerintele de timp
19 const TickType_t TAU_MIN = pdMS_TO_TICKS(1000);  // 1s: garda de
      siguranta dupa S1
20 const TickType_t TAU_MAX = pdMS_TO_TICKS(5000);  // 5s: timp maxim
      de stationare la S2
21
22 const uint32_t TAU_OPEN_MS  = 30;               // timp deschidere
      liniara
23 const uint32_t TAU_CLOSE_MS = 30;               // timp inchidere
      liniara
24
25 // Unghiuri
26 const int ANGLE_OPEN  = 90;     // Deschis (Utilitate)
27 const int ANGLE_CLOSE = 180;    // Inchis (Siguranta)
28
29 // Evenimente
30 typedef enum {
31   EVT_S1_PRESENT,         // S1 < PRAG_CM
32   EVT_S2_PRESENT,         // S2 < PRAG_CM
33   EVT_S2_CLEAR,           // S2 > (PRAG_CM + HYST_CM)
34   EVT_TAU_MIN_EXPIRED,    // garda de 1s s-a incheiat
35   EVT_TAU_MAX_EXPIRED,    // stationare prelungita la S2
36   EVT_OPEN_DONE,          // confirmare Servo deschis
37   EVT_CLOSE_DONE          // confirmare Servo inchis
38 } Event_t;
39
40 // Comenzi servo (pentru miscare liniara)
41 typedef struct {
42   int targetAngle;
43   uint32_t durationMs;
44 } ServoCmd_t;
45
46 // Obiectele utilizate
47 static QueueHandle_t qEvents;
48 static QueueHandle_t qServo;
49
50 static TimerHandle_t tTauMax;
51 static TimerHandle_t tTauMin;
52 static TimerHandle_t tOpenDone;
53 static TimerHandle_t tCloseDone;
54
55 static Servo barrier;
```

```
56
57 // Stari
58 static volatile bool s2Present = false;           // masina este in
       zona S2
59 static volatile bool tauMinActive = false;        // suntem in
      garda obligatorie de TAU_MIN
60 static volatile bool ignoreUntilClearAfterTauMax = false; // pentru
       recuperare dupa TAU_MAX
61
62 // Citire distanta senzor ultrasonic
63 static float readDistanceCm(int trigPin, int echoPin) {
64   digitalWrite(trigPin, LOW);
65   digitalWrite(trigPin, HIGH);
66   digitalWrite(trigPin, LOW);
67 // blocare necesara
68   unsigned long dur = pulseIn(echoPin, HIGH, 30000); // timeout 30
       ms
69   if (dur == 0) return -1.0f;
70   return (dur * 0.0343f) / 2.0f;
71 }
72
73 // Callback-uri pentru timere (toate Callback-urile trimit
      evenimente in coada qEvents, in mod asincron)
74 static void cbTauMax(TimerHandle_t) {
75   Event_t ev = EVT_TAU_MAX_EXPIRED;
76   xQueueSend(qEvents, &ev, 0);
77 }
78 static void cbTauMin(TimerHandle_t) {
79   Event_t ev = EVT_TAU_MIN_EXPIRED;
80   xQueueSend(qEvents, &ev, 0);
81 }
82 static void cbOpenDone(TimerHandle_t) {
83   Event_t ev = EVT_OPEN_DONE;
84   xQueueSend(qEvents, &ev, 0);
85 }
86 static void cbCloseDone(TimerHandle_t) {
87   Event_t ev = EVT_CLOSE_DONE;
88   xQueueSend(qEvents, &ev, 0);
89 }
90
91 // Task-ul servomotorului (miscare liniara)
92 static void taskServo(void*) {
93   barrier.attach(PIN_SERVO);
```

```
94    barrier.write(ANGLE_OPEN); // initial deschis
95
96    ServoCmd_t cmd;
97    while (1) {
98      if (xQueueReceive(qServo, &cmd, portMAX_DELAY) == pdPASS) {
99        int start = barrier.read();
100       int target = cmd.targetAngle;
101
102       uint32_t T = cmd.durationMs;
103       if (T < 5) T = 5;
104
105       const uint32_t stepMs = 1;
106       uint32_t steps = T / stepMs;
107       if (steps < 5) steps = 5;
108
109       // miscarea liniara
110       for (uint32_t i = 1; i <= steps; i++) {
111         int ang = start + (int)((long)(target - start) * (long)i /
              (long)steps);
112         barrier.write(ang);
113         vTaskDelay(pdMS_TO_TICKS(stepMs));
114       }
115       barrier.write(target);
116     }
117   }
118 }
119
120 // Helpers pentru functii
121 static void requestClose() {
122   ServoCmd_t c{ANGLE_CLOSE, TAU_CLOSE_MS};
123   xQueueSend(qServo, &c, 0);
124   xTimerStop(tOpenDone, 0);
125   xTimerStop(tCloseDone, 0);
126   xTimerStart(tCloseDone, 0);
127 }
128
129 static void requestOpen() {
130   ServoCmd_t c{ANGLE_OPEN, TAU_OPEN_MS};
131   xQueueSend(qServo, &c, 0);
132   xTimerStop(tOpenDone, 0);
133   xTimerStop(tCloseDone, 0);
134   xTimerStart(tOpenDone, 0);
135 }
```

```
136
137 // Task de monitorizare a senzorilor
138 static void taskSensors(void*) {
139   pinMode(PIN_TRIG_S1, OUTPUT);
140   pinMode(PIN_ECHO_S1, INPUT);
141   pinMode(PIN_TRIG_S2, OUTPUT);
142   pinMode(PIN_ECHO_S2, INPUT);
143
144   bool s1Prev = false;
145   bool s2PrevPresent = false;
146
147   while (1) {
148     float d1 = readDistanceCm(PIN_TRIG_S1, PIN_ECHO_S1);
149     float d2 = readDistanceCm(PIN_TRIG_S2, PIN_ECHO_S2);
150
151     bool s1 = (d1 > 0 && d1 < PRAG_CM);
152
153     bool s2PresentNow = (d2 > 0 && d2 < PRAG_CM);
154     bool s2ClearNow   = (d2 < 0) || (d2 > (PRAG_CM + HYST_CM)); //
          S2 cu histerezis
155
156     // S1: eveniment pe front pozitiv (apropierea unui tren)
157     if (s1 && !s1Prev) {
158       Event_t ev = EVT_S1_PRESENT;
159       xQueueSend(qEvents, &ev, 0);
160     }
161     s1Prev = s1;
162
163     // S2: prezent pe front pozitiv
164     if (s2PresentNow && !s2PrevPresent) {
165       Event_t ev = EVT_S2_PRESENT;
166       xQueueSend(qEvents, &ev, 0);
167       s2PrevPresent = true;
168     }
169
170     // S2: "clear" pe front pozitiv
171     if (s2ClearNow && s2PrevPresent) {
172       Event_t ev = EVT_S2_CLEAR;
173       xQueueSend(qEvents, &ev, 0);
174       s2PrevPresent = false;
175     }
176
177     // afisarea distantelor
```

```
178     Serial.print("S1="); Serial.print(d1, 1); Serial.print("cm  ");
179     Serial.print("S2="); Serial.print(d2, 1); Serial.println("cm");
180
181     vTaskDelay(pdMS_TO_TICKS(60));
182   }
183 }
184
185 // Task-ul de control
186 static void taskControl(void*) {
187   Event_t ev;
188
189   Serial.println("INIT: Bariera OPEN.");
190   Serial.println("Reguli: S1 -> inchide + tine inchis TAU_MIN=1s
        indiferent. Dupa TAU_MIN: daca S2 prezent, tine inchis pana
        pleaca sau TAU_MAX=5s.");
191
192   while (1) {
193     if (xQueueReceive(qEvents, &ev, portMAX_DELAY) == pdPASS) {
194
195       switch (ev) {
196
197         // S1: inchiderea de siguranta, garda tau_min
198         case EVT_S1_PRESENT:
199           Serial.println("S1 < 5cm -> INCHID bariera si respect
                TAU_MIN=1s.");
200           requestClose();
201
202           tauMinActive = true;
203           xTimerStop(tTauMin, 0);
204           xTimerChangePeriod(tTauMin, TAU_MIN, 0);
205           xTimerStart(tTauMin, 0);
206           break;
207
208         // S2: tren in zona de control
209         case EVT_S2_PRESENT:
210           if (ignoreUntilClearAfterTauMax) {
211             Serial.println("IGNOR: TAU_MAX a expirat, astept S2
                  CLEAR.");
212             break;
213           }
214
215           s2Present = true;
```

```
216           Serial.println("S2 < 5cm -> masina in zona: bariera
                 trebuie inchisa, pornesc TAU_MAX=5s.");
217           requestClose();
218
219           // porneste TAU_MAX doar daca nu este deja activat de
                 expirarea TAU_MIN
220           if (!tauMinActive) {
221             xTimerStop(tTauMax, 0);
222             xTimerChangePeriod(tTauMax, TAU_MAX, 0);
223             xTimerStart(tTauMax, 0);
224           }
225           break;
226
227         // S2: trenul a plecat
228         case EVT_S2_CLEAR:
229           Serial.println("S2 > 5cm -> masina a plecat.");
230           s2Present = false;
231           ignoreUntilClearAfterTauMax = false;
232           xTimerStop(tTauMax, 0);
233
234           if (!tauMinActive) {
235             Serial.println("TAU_MIN nu e activ -> RIDIC bariera.");
236             requestOpen();
237           } else {
238             Serial.println("TAU_MIN activ -> bariera ramane inchisa
                   pana expira TAU_MIN.");
239           }
240           break;
241
242         // expirarea TAU_MIN
243         case EVT_TAU_MIN_EXPIRED:
244           tauMinActive = false;
245           Serial.println("TAU_MIN expirat -> verific S2.");
246
247           if (s2Present) {
248             Serial.println("S2 prezent -> raman inchis si pornesc
                   TAU_MAX.");
249             if (xTimerIsTimerActive(tTauMax) == pdFALSE) {
250               xTimerChangePeriod(tTauMax, TAU_MAX, 0);
251               xTimerStart(tTauMax, 0);
252             }
253           } else {
254             Serial.println("S2 nu e prezent -> RIDIC bariera.");
```

```
255             requestOpen();
256           }
257           break;
258
259         // expirarea TAU_MAX: fortarea plecarii trenului
260         case EVT_TAU_MAX_EXPIRED:
261           if (s2Present) {
262             Serial.println("Masina a stat TAU_MAX=5s la S2 -> RIDIC
                  bariera.");
263             requestOpen();
264
265             // setare flag pentru a ignora reinchiderea imediata
266             ignoreUntilClearAfterTauMax = true;
267             s2Present = false;
268           }
269           break;
270
271         // confirmare
272         case EVT_CLOSE_DONE:
273           Serial.println("--- Bariera CLOSED complet (30ms).");
274           break;
275
276         case EVT_OPEN_DONE:
277           Serial.println("--- Bariera OPEN complet (30ms).");
278           break;
279
280         default:
281           break;
282       }
283     }
284   }
285 }
286
287 void setup() {
288   Serial.begin(9600);
289
290   // creare cozi
291   qEvents = xQueueCreate(12, sizeof(Event_t));
292   qServo  = xQueueCreate(5, sizeof(ServoCmd_t));
293
294   // creare timere
295   tTauMin    = xTimerCreate("tMin", TAU_MIN, pdFALSE, NULL,
        cbTauMin);
```

```
296   tTauMax    = xTimerCreate("tMax", TAU_MAX, pdFALSE, NULL,
          cbTauMax);
297   tOpenDone  = xTimerCreate("tOpen", pdMS_TO_TICKS(TAU_OPEN_MS),
          pdFALSE, NULL, cbOpenDone);
298   tCloseDone = xTimerCreate("tClose", pdMS_TO_TICKS(TAU_CLOSE_MS),
          pdFALSE, NULL, cbCloseDone);
299
300   // creare task-uri (prioritate: CTRL > SERVO > SENS)
301   xTaskCreate(taskServo,   "SERVO",  256, NULL, 2, NULL);
302   xTaskCreate(taskSensors, "SENS",   384, NULL, 1, NULL);
303   xTaskCreate(taskControl, "CTRL",   512, NULL, 3, NULL);
304
305   vTaskStartScheduler();
306 }
307
308 void loop() {}
```

4 Testarea aplicației și validarea soluției propuse

Testarea aplicației a fost realizată pe etape, urmărind principiul dezvoltării modulare: am validat fiecare componentă hardware/software înainte de integrarea sa în sistemul complex bazat pe FreeRTOS.

În faza inițială a proiectului, am urmărit asigurarea funcționării corecte a perifericelor critice:

- **Placa de Bază (Arduino Mega):** Verificarea funcționalității de bază și a mediului FreeRTOS.
- **Senzorii Ultrasonici (S1 și S2):** Senzorii au fost testați inițial cu coduri simple pentru a valida citirea precisă a distanței (`readDistanceCm`) și a determina pragurile optime de detecție (`PRAG_CM = 5.0cm`) și histerezisul (`HYST_CM = 1.0cm`).
- **Actuatorul (Servomotorul):** Servomotorul a fost testat individual pentru a confirma funcționarea `Servo.attach` și pentru a mapa corect unghiurile de `ANGLE_OPEN` (90°) și `ANGLE_CLOSE` (180°).

Validarea soluției propuse vizează confirmarea faptului că logica din `taskControl` și mecanismele FreeRTOS (timere, cozi) îndeplinesc cerințele de Siguranță și Utilitate. Am testat fizic diferite secvențe de evenimente simulate pe coada `qEvents` pentru a acoperi cazurile critice în timp real.

1. **Testul de Siguranță (Apropiere Tren):**
   - **Secvența:** `EVT_S1_PRESENT` → `EVT_CLOSE_DONE`.
   - **Așteptat:** Bariera începe imediat închiderea, iar timpul necesar închiderii este `TAU_CLOSE_MS` (30ms), confirmat de `EVT_CLOSE_DONE`.
2. **Testul de Gardă (Siguranță τ_{min}):**
   - **Secvența:** `EVT_S1_PRESENT` → `EVT_S2_CLEAR` (rapid, în $< 1s$) → `EVT_TAU_MIN_EXPIRED`.
   - **Așteptat:** După `EVT_S2_CLEAR`, bariera rămâne închisă deoarece `tauMinActive = true`. Deschiderea se inițiază doar după expirarea `EVT_TAU_MIN_EXPIRED` ($1s$), garantând timpul minim de siguranță.
3. **Testul de Utilitate (Staționare τ_{max}):**
   - **Secvența:** `EVT_S1_PRESENT` → τ_{min} Expired → `EVT_S2_PRESENT` (trenul stă) → `EVT_TAU_MAX_EXPIRED` ($5s$).

- **Așteptat:** Trenul este forțat să plece, iar bariera să se deschidă (Utilitate). Starea `ignoreUntilClearAfterTauMax` este activată pentru a preveni reînchiderea imediată, răspunzând astfel cerinței de a elibera traficul rutier după depășirea timpului maxim de staționare.

4.1 Rezultatele execuției

În imaginile de mai jos se pot observa rezultatele rulării aplicației.

```
S1=44.4cm  S2=2.9cm
Masina a stat TAU_MAX=5s la S2 -> RIDIC bariera.
--- Bariera OPEN complet (30ms).
S1=42.8cm  S2=3.0cm
S1=42.8cm  S2=3.1cm
S1=42.7cm  S2=3.1cm
S1=39.6cm  S2=3.1cm
S1=42.1cm  S2=3.1cm
S1=43.6cm  S2=4.6cm
S1=43.5cm  S2=4.3cm
S1=40.8cm  S2=3.1cm
S1=40.9cm  S2=4.7cm
S1=40.5cm  S2=3.1cm
S1=46.1cm  S2=6.0cm
S1=44.3cm  S2=4.7cm
S1=46.1cm  S2=6.0cm
S1=44.6cm  S2=4.7cm
S1=45.0cm  S2=4.4cm
S1=45.3cm  S2=4.0cm
S1=44.0cm  S2=11.3cm
S2 > 5cm -> masina a plecat.
TAU_MIN nu e activ -> RIDIC bariera.
--- Bariera OPEN complet (30ms).
```

```
S1 < 5cm -> INCHID bariera si respect TAU_MIN=1s.
--- Bariera CLOSED complet (30ms).
S1=3.4cm  S2=49.6cm
S1=2.6cm  S2=49.2cm
S1=2.3cm  S2=48.8cm
S1=2.3cm  S2=30.0cm
S1=3.5cm  S2=29.4cm
S1=3.6cm  S2=56.4cm
S1=2.7cm  S2=52.7cm
S1=2.7cm  S2=56.5cm
S1=2.7cm  S2=56.5cm
S1=2.7cm  S2=56.5cm
S1=2.7cm  S2=56.1cm
S1=2.7cm  S2=56.5cm
S1=2.7cm  S2=-1.0cm
S1=2.7cm  S2=-1.0cm
S1=2.7cm  S2=-1.0cm
S1=2.7cm  S2=-1.0cm
S1=2.7cm  S2=-1.0cm
S1=2.6cm  S2=-1.0cm
TAU_MIN expirat -> verific S2.
S2 nu e prezent -> RIDIC bariera.
--- Bariera OPEN complet (30ms).
```

```
S1 < 5cm -> INCHID bariera si respect TAU_MIN=1s.
--- Bariera CLOSED complet (30ms).
S1=43.1cm  S2=29.8cm
S1=42.8cm  S2=48.1cm
S1=9.6cm  S2=47.5cm
S1=43.1cm  S2=46.8cm
S1=42.1cm  S2=29.6cm
S1=41.8cm  S2=6.2cm
S1=41.4cm  S2=4.4cm
S2 < 5cm -> masina in zona: bariera trebuie inchisa, pornesc TAU_MAX=5s.
--- Bariera CLOSED complet (30ms).
S1=46.1cm  S2=5.8cm
S1=47.0cm  S2=6.0cm
S1=49.2cm  S2=4.4cm
S1=50.1cm  S2=6.0cm
S1=50.0cm  S2=4.7cm
S1=40.7cm  S2=4.7cm
S1=42.1cm  S2=6.0cm
S1=42.4cm  S2=3.1cm
S1=42.5cm  S2=2.9cm
S1=44.1cm  S2=3.1cm
S1=45.5cm  S2=3.1cm
S1=48.8cm  S2=3.1cm
TAU_MIN expirat -> verific S2.
S2 prezent -> raman inchis pana S2 CLEAR sau expira TAU_MAX.
```

Soluția implementată, utilizând task-uri separate și sincronizare prin cozi de evenimente, a demonstrat o separare clară a preocupărilor (I/O în `taskSensors`, logica FSM în `taskControl`, acționarea liniară în `taskServo`).

Utilizarea timerelor software FreeRTOS s-a dovedit esențială pentru a modela cu precizie constrângerile temporale ($\tau_{min}, \tau_{max}, \tau_{open}, \tau_{close}$) fără a bloca task-ul critic de control. Prin alegerea priorităților, am asigurat că deciziile de Siguranță primesc cea mai rapidă execuție posibilă în mediul RTOS. Programul răspunde satisfăcător și respectă întocmai cerințele critice specificate.

5 Bibliografie

Bibliografie

[1] **Lamie, Edward L.** *Real-Time Embedded Multithreading Using ThreadX: Third Edition*. Editura Elsevier, 2019.

[2] **FreeRTOS**. *Real Time Kernel Overview*. Link: `https://www.freertos.org/Documentation/00-Overview`

[3] **YouTube**. *Video referinta Proiect Similar (Gate Opener)*. Link: `https://www.youtube.com/watch?v=F8WAsUUT30E`

[4] **YouTube**. *Video referinta Proiect Similar (Railway Crossing Automatic Barrier)*. Link: `https://www.youtube.com/watch?v=LJ1xLmmuTUs`